

pset5

March 6, 2026

1 PSET 5: Calculations

Harsh Dadhich, 505960992

1.1 Part 2: Linear Programming Optimization (Manufacturing Levels)

- **Objective:** Maximize net rev
- **Constraints:** Production $\leq$ 25,000; Sales $\leq$ exp demand

```
[15]: %pip install pulp
```

Requirement already satisfied: pulp in
/home/hdadhich/ENGR-213/venv/lib/python3.13/site-packages (3.3.0)
Note: you may need to restart the kernel to use updated packages.

```
[16]: # Part 2: Linear Programming , Optimal Manufacturing Level
import pulp

# Exp Demand (weighted average of Mid Price scenarios)
# Demand scenarios: 5k (0.2), 15k (0.3), 25k (0.5)
Expected_Demand = 5000 * 0.2 + 15000 * 0.3 + 25000 * 0.5
print(f"Expected Demand: {Expected_Demand:,.0f} units\n")

# Init PuLP Maximization Problem
prob = pulp.LpProblem("Smart_Water_Bottle_Production", pulp.LpMaximize)

# Decision Variables (non-negative)
Manufactured_Units = pulp.LpVariable("Manufactured", lowBound=0, cat="Continuous")
Sold_Units = pulp.LpVariable("Sold", lowBound=0, cat="Continuous")
Inventory_Units = pulp.LpVariable("Inventory", lowBound=0, cat="Continuous")

# Objective: Maximize Net Revenue
# Revenue from sales - Mfg cost - Holding cost for unsold inventory
prob += (70 * Sold_Units) - (30 * Manufactured_Units) - (
    5 * Inventory_Units
), "Net_Revenue"

# Constraints
```

```

prob += Manufactured_Units <= 25000, "Capacity"
prob += Sold_Units <= Expected_Demand, "Sales_Limit_Demand"
prob += Sold_Units <= Manufactured_Units, "Sales_Limit_Production"
prob += Inventory_Units == Manufactured_Units - Sold_Units, "Inventory_Balance"

# Solve
prob.solve()

# Results
print(f"Status: {pulp.LpStatus[prob.status]}")
print(f"Optimal Manufacturing Level: {int(Manufactured_Units.varValue)} units")
print(f"Units Sold: {int(Sold_Units.varValue)} units")
print(f"Inventory: {int(Inventory_Units.varValue)} units")
print(f"Total Net Revenue: ${pulp.value(prob.objective):,.0f}")

```

Expected Demand: 18,000 units

Welcome to the CBC MILP Solver

Version: 2.10.10

Build Date: Sep 26 2023

command line - /home/hdadhich/ENGR-213/venv/lib/python3.13/site-packages/pulp/apis/./solverdir/cbc/linux/arm64/cbc
/tmp/e2d17194b842423fbb73142660a330e7-pulp.mps -max -timeMode elapsed -branch
-printingOptions all -solution /tmp/e2d17194b842423fbb73142660a330e7-pulp.sol
(default strategy 1)

At line 2 NAME MODEL

At line 3 ROWS

At line 9 COLUMNS

At line 20 RHS

At line 25 BOUNDS

At line 26 ENDDATA

Problem MODEL has 4 rows, 3 columns and 7 elements

Coin0008I MODEL read with 0 errors

Option for timeMode changed from cpu to elapsed

Presolve 2 (-2) rows, 3 (0) columns and 5 (-2) elements

0 Obj -0 Dual inf 69.999999 (1)

1 Obj 720000

Optimal - objective value 720000

After Postsolve, objective 720000, infeasibilities - dual 0 (0), primal 0 (0)

Optimal objective 720000 - 1 iterations time 0.002, Presolve 0.00

Option for printingOptions changed from normal to all

Total time (CPU seconds): 0.00 (Wallclock seconds): 0.00

Status: Optimal

Optimal Manufacturing Level: 18000 units

Units Sold: 18000 units

Inventory: 0 units

Total Net Revenue: \$720,000

1.2 Part 3: Inventory Planning with the Newsvendor Model

Here we account for uncertainty using the Newsvendor model

- **Rule:** We choose the smallest demand level where the cumulative probability $\geq$ critical ratio (CR)

```
[17]: # Part 3: Newsvendor Model - Optimal Stock Level
Price = 70
Cost = 30
Holding = 5

# Cost of Underage: lost profit per unit if we could have sold one more
Cu = Price - Cost
# Cost of Overage: cost + holding per unit of unsold inventory
Co = Cost + Holding
# Critical Ratio
CR = Cu / (Cu + Co)

print(f"Cost of Underage (Cu): ${Cu}")
print(f"Cost of Overage (Co): ${Co}")
print(f"Critical Ratio (CR): {CR:.4f}\n")

# Demand scenarios: (demand, probability)
scenarios = [
    {"demand": 5000, "prob": 0.2},
    {"demand": 15000, "prob": 0.3},
    {"demand": 25000, "prob": 0.5},
]
# Sort by demand (ascending)
scenarios_sorted = sorted(scenarios, key=lambda x: x["demand"])

# Find smallest demand level where cumul prob >= CR
cumulative_prob = 0
Optimal_Stock_Level = None
for s in scenarios_sorted:
    cumulative_prob += s["prob"]
    if cumulative_prob >= CR:
        Optimal_Stock_Level = s["demand"]
        break

print(f"Optimal Stock Level (Newsvendor): {Optimal_Stock_Level} units")
```

Cost of Underage (Cu): \$40

Cost of Overage (Co): \$35

Critical Ratio (CR): 0.5333

Optimal Stock Level (Newsvendor): 25000 units

```
[18]: # Expected Profit using Optimal Stock Level from Part 3
Stock = Optimal_Stock_Level # 25,000 from Newsvendor

Expected_Profit = 0
for s in scenarios_sorted:
    demand = s["demand"]
    prob = s["prob"]
    if demand >= Stock:
        Sold = Stock
        Unsold = 0
    else:
        Sold = demand
        Unsold = Stock - demand
    Profit = (Sold * Price) - (Stock * Cost) - (Unsold * Holding)
    Expected_Profit += prob * Profit

print(f"Expected Profit (with Newsvendor optimal stock): ${Expected_Profit:,.0f}")
```

Expected Profit (with Newsvendor optimal stock): \$475,000